Day 1-2: Core Concepts - Complete Study Material

---

1. What is System Design?

Definition

System Design is the process of defining the architecture, components, modules, interfaces, and data flow of a system to satisfy specified requirements. It's about making decisions on how to build scalable, reliable, and maintainable systems.

Why Do We Need System Design?

Imagine you're building a simple todo app for yourself. You could:

- Store data in a local file
- Run it on your laptop
- No need to worry about multiple users

But what if 1 million people want to use your app?

Now you need to think about:

- Where to store all that data?
- How to handle thousands of requests per second?
- What happens if a server crashes?
- How to keep the app fast for users worldwide?

This is where System Design comes in.

Real-World Example: Instagram

When Instagram launched in 2010, it was 2 engineers handling a few thousand users. By 2012, they had 30 million users. Their initial simple architecture couldn't handle this:

```
Initial (Simple):
[Mobile App] → [Single Server] → [Single Database]
```

```
Problem: Single server couldn't handle millions of requests

Evolved (Scalable):
[Mobile Apps] → [Load Balancer] → [Multiple Servers] → [Database Cluster]
                        ↓
                    [CDN for images]
                        ↓
                    [Cache Layer]
```

Types of System Design

1. High-Level Design (HLD)

- What: Bird's eye view of the entire system
- Focus: Components, their interactions, data flow
- Output: Architecture diagrams, component list
- Example Question: "Design Twitter" → Draw boxes for Feed Service, User Service, Database, Cache, etc.

2. Low-Level Design (LLD)

- What: Detailed design of individual components
- Focus: Classes, methods, database schemas, APIs
- Output: Class diagrams, API specifications, database schemas
- Example Question: "Design a Rate Limiter" → Write actual classes and algorithms

Key Goals of System Design

Goal	What It Means	Example
Scalability	Handle growth (users, data, traffic)	Netflix handles 250M+ subscribers
Reliability	System works correctly even when things fail	AWS promises 99.99% uptime
Availability	System is accessible when needed	Google Search is always up
Maintainability	Easy to update, debug, and extend	Well-documented microservices

Performance	Fast response times	Google Search < 200ms
Cost-Effectiveness	Optimize infrastructure costs	Use caching to reduce DB load

The System Design Thought Process

When approaching any design problem, think in this order:

```
1. WHAT are we building?
   └── Functional requirements (features)
   └── Non-functional requirements (scale, latency, availability)

2. HOW MUCH scale do we need?
   └── Users, requests/second, data size
   └── Read-heavy or write-heavy?

3. WHAT components do we need?
   └── Servers, databases, caches, queues, etc.

4. HOW do they connect?
   └── Data flow, APIs, protocols

5. WHAT can go wrong?
   └── Single points of failure
   └── Bottlenecks
```

---

2. Client-Server Architecture

What is Client-Server Architecture?

It's a computing model where:

- Client: Requests services/resources (your browser, mobile app)
- Server: Provides services/resources (web server, API server)

They communicate over a network (usually the internet).

```
┌──────────────┐          Request          ┌──────────────┐
│              │  ──────────────────────>  │              │
│   CLIENT     │                           │   SERVER     │
│  (Browser)   │  <──────────────────────  │  (Web App)   │
│              │          Response         │              │
```

```
└──────────────┘                                  └──────────────┘
```

How the Web Works - Step by Step

Let's trace what happens when you type `https://www.google.com` in your browser:

Step 1: DNS Lookup (We'll cover this in detail later)

Your browser needs to find Google's IP address.

```
"www.google.com" → DNS → "142.250.190.68"
```

Step 2: TCP Connection

Your browser establishes a connection to Google's server.

```
Browser ⟷ Google Server (TCP Handshake)
```

Step 3: HTTP Request

Browser sends a request for the homepage.

```
GET / HTTP/1.1
Host: www.google.com
User-Agent: Chrome/120.0
Accept: text/html
```

Step 4: Server Processing

Google's server:

1. Receives the request
2. Processes it (runs code, queries databases)
3. Generates HTML response

Step 5: HTTP Response

Server sends back the response.

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 45678

<!DOCTYPE html>
<html>...Google's homepage HTML...</html>
```

Step 6: Rendering

Browser parses HTML, downloads CSS/JS/images, renders the page.

Types of Servers

Server Type	Purpose	Examples
Web Server	Serve static files (HTML, CSS, JS, images)	Nginx, Apache
Application Server	Run business logic, process requests	Node.js, Tomcat, uWSGI
Database Server	Store and retrieve data	MySQL, PostgreSQL, MongoDB
Cache Server	Store frequently accessed data in memory	Redis, Memcached
File Server	Store and serve files	AWS S3, Google Cloud Storage

Modern Architecture Patterns

Simple Architecture (Small Apps)

```
[Client] → [Web Server + App Logic] → [Database]
```

Separated Frontend/Backend

```
[Client] → [Web Server (Nginx)] → [API Server (Node.js)] → [Database]
              serving static files     handling logic
```

Microservices Architecture

```
                    ┌─→ [User Service] → [User DB]
                    │
[Client] → [API Gateway] ─→ [Order Service] → [Order DB]
                    │
                    └─→ [Payment Service] → [Payment DB]
```

Stateless vs Stateful Servers

Stateless Server

- Doesn't remember previous requests
- Each request is independent
- Scalable: Any server can handle any request
- Session data stored externally (database, Redis)

```
Request 1 → Server A → "Who are you?" → Check session in Redis
Request 2 → Server B → "Who are you?" → Check session in Redis
(Same user, different servers, still works!)
```

Stateful Server

- Remembers previous requests
- Stores session data locally
- Problem: User must always hit the same server

```
Request 1 → Server A → Login → Session stored in Server A's memory
Request 2 → Server B → "Who are you?" → No session! Logged out!
```

Best Practice: Build stateless servers for scalability.

---

3. HTTP/HTTPS Basics

What is HTTP?

HTTP (HyperText Transfer Protocol) is the foundation of data communication on the web. It defines how messages are formatted and transmitted between clients and servers.

HTTP Request Structure

Every HTTP request has these parts:

```
┌──────────────────────────────────────────────────────────────┐
│ POST /api/users HTTP/1.1                    ← Request Line
├──────────────────────────────────────────────────────────────┤
│ Host: api.example.com                       ← Headers
│ Content-Type: application/json
│ Authorization: Bearer eyJhbGc...
│ Accept: application/json
│ Content-Length: 42
```

```
├──────────────────────────────────────────────────────────────┤
│                                                ← Empty Line
├──────────────────────────────────────────────────────────────┤
│ {"name": "John", "email": "john@test.com"} ← Body (optional)
└──────────────────────────────────────────────────────────────┘
```

HTTP Methods (Verbs)

Method	Purpose	Idempotent?	Has Body?	Example
GET	Retrieve data	Yes	No	Get user profile
POST	Create new resource	No	Yes	Create new user
PUT	Replace entire resource	Yes	Yes	Update user completely
PATCH	Partial update	No	Yes	Update just email
DELETE	Remove resource	Yes	Usually No	Delete user
HEAD	GET without body (metadata only)	Yes	No	Check if file exists
OPTIONS	Get allowed methods	Yes	No	CORS preflight

Idempotent: Same request multiple times = same result

- `DELETE /users/5` three times → user 5 is deleted (same result)
- `POST /users` three times → 3 users created (NOT idempotent)

HTTP Response Structure

```
┌──────────────────────────────────────────────────────────────┐
│ HTTP/1.1 200 OK                                ← Status Line
├──────────────────────────────────────────────────────────────┤
│ Content-Type: application/json                 ← Headers
│ Cache-Control: max-age=3600
│ X-Request-Id: abc123
```

```
│ Content-Length: 89
├──────────────────────────────────────────────────────────────────────┤
│                                                        ← Empty Line
├──────────────────────────────────────────────────────────────────────┤
│ {"id": 1, "name": "John", "email": "..."}   ← Body
└──────────────────────────────────────────────────────────────────────┘
```

HTTP Status Codes (IMPORTANT!)

Memorize these - they're asked in interviews:

1xx - Informational

Code	Meaning
100	Continue
101	Switching Protocols (e.g., upgrading to WebSocket)

2xx - Success

Code	Meaning	When to Use
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST (resource created)
204	No Content	Successful DELETE (nothing to return)

3xx - Redirection

Code	Meaning	When to Use
301	Moved Permanently	URL changed forever (SEO)
302	Found (Temporary Redirect)	Temporary redirect
304	Not Modified	Use cached version

4xx - Client Errors

Code	Meaning	When to Use
400	Bad Request	Invalid syntax, missing fields

401	Unauthorized	Not authenticated (login required)
403	Forbidden	Authenticated but not allowed
404	Not Found	Resource doesn't exist
405	Method Not Allowed	POST to GET-only endpoint
409	Conflict	Resource conflict (duplicate)
422	Unprocessable Entity	Validation failed
429	Too Many Requests	Rate limited

5xx - Server Errors

Code	Meaning	When to Use
500	Internal Server Error	Unhandled exception
502	Bad Gateway	Upstream server failed
503	Service Unavailable	Server overloaded/maintenance
504	Gateway Timeout	Upstream server timeout

Common HTTP Headers

Request Headers

Header	Purpose	Example
`Host`	Target server	`Host: api.example.com`
`Authorization`	Authentication	`Authorization: Bearer <token>`
`Content-Type`	Body format	`Content-Type: application/json`
`Accept`	Expected response format	`Accept: application/json`
`User-Agent`	Client info	`User-Agent: Chrome/120.0`

`Cookie`	Send cookies	`Cookie: session=abc123`
`Cache-Control`	Caching behavior	`Cache-Control: no-cache`

Response Headers

Header	Purpose	Example
`Content-Type`	Response format	`Content-Type: application/json`
`Set-Cookie`	Set cookies	`Set-Cookie: session=abc; HttpOnly`
`Cache-Control`	Cache instructions	`Cache-Control: max-age=3600`
`Location`	Redirect URL	`Location: /new-page`
`X-RateLimit-Remaining`	Rate limit info	`X-RateLimit-Remaining: 95`

HTTP vs HTTPS

HTTP (Port 80)

- Data sent in plain text
- Anyone on the network can read it
- Never use for sensitive data

```
[Browser] ——— "password=secret123" ———> [Server]
                    ↑
            Hacker can intercept and read!
```

HTTPS (Port 443)

- Data is encrypted using TLS/SSL
- Even if intercepted, data is unreadable
- Always use for production

```
[Browser] ——— "xK9#mP2$vL..." (encrypted) ———> [Server]
                    ↑
            Hacker sees gibberish!
```

TLS/SSL Handshake (Simplified)

When you connect via HTTPS:

```
1. Client Hello
   Browser → Server: "I support these encryption methods"

2. Server Hello + Certificate
   Server → Browser: "Let's use AES-256. Here's my certificate."

3. Certificate Verification
   Browser checks: Is this certificate valid? Signed by trusted CA?

4. Key Exchange
   Browser & Server agree on a shared secret key

5. Encrypted Communication
   All data encrypted with the shared key
```

HTTP Versions

Version	Year	Key Features
HTTP/1.0	1996	One request per connection
HTTP/1.1	1997	Keep-alive, pipelining, Host header
HTTP/2	2015	Multiplexing, server push, header compression
HTTP/3	2022	Uses QUIC (UDP-based), faster

HTTP/1.1 vs HTTP/2

```
HTTP/1.1 (Sequential):
Request 1 ──────────> Response 1
                      Request 2 ──────────> Response 2
                                            Request 3 ──────────> Response 3

HTTP/2 (Multiplexed):
Request 1 ─┐
Request 2 ─┼──────────> All responses arrive together
Request 3 ─┘            (on single connection)
```

The Core Difference

HTTP/1.1: One at a Time (Sequential)

In HTTP/1.1, even with keep-alive connections, requests are processed one after another. You must wait for a response before the next request can be processed on that connection.

HTTP/2: All at Once (Multiplexed)

In HTTP/2, multiple requests can be sent simultaneously on a single connection, and responses come back as soon as they're ready - interleaved on the same connection.

Real-Life Analogy: The Restaurant

HTTP/1.1 = Single Waiter, One Order at a Time

Imagine a restaurant with one waiter who can only handle one order at a time:

```
You (Customer)                                Kitchen
    |                                           |
    |—— "I want soup" ————————————————————>|
    |     (waiter goes to kitchen)              |
    |     (waiter waits for soup)               |
    |     (waiter brings soup back)             |
    |<————————————  Soup ————————————————————|  (3 minutes)
    |                                           |
    |—— "I want salad" ———————————————————>|
    |     (waiter goes to kitchen)              |
    |     (waiter waits for salad)              |
    |     (waiter brings salad back)            |
    |<————————————— Salad ————————————————————|  (2 minutes)
    |                                           |
    |—— "I want steak" ———————————————————>|
    |     (waiter goes to kitchen)              |
    |     (waiter waits for steak)              |
    |     (waiter brings steak back)            |
    |<————————————— Steak ————————————————————|  (5 minutes)
    |                                           |
    |         TOTAL TIME: 10 minutes            |
```

You wait 10 minutes total because each order is handled sequentially.

HTTP/2 = Smart Waiter with a Tray

Now imagine a waiter who takes all orders at once and brings items back as they're ready:

```
You (Customer)                                Kitchen
    |                                           |
    |—— "I want soup" ————————————————————————>|
    |—— "I want salad" ———————————————————————>|   All orders sent
    |—— "I want steak" ———————————————————————>|   immediately!
    |                                           |
    |     (kitchen works on all 3 in parallel)  |
    |                                           |
    |<————————  Salad    ———————————————————————|  (ready in 2 min)
    |<————————    Soup   ———————————————————————|  (ready in 3 min)
    |<————————    Steak  ———————————————————————|  (ready in 5 min)
    |                                           |
    |         TOTAL TIME: 5 minutes             |
    |    (time of the slowest item only!)       |
```

You wait only 5 minutes because everything was prepared in parallel!

Technical Explanation with a Real Webpage

Scenario: Loading a Webpage

When you visit a webpage, your browser needs to download:

- `index.html` (10 KB, 50ms to generate)
- `styles.css` (5 KB, 10ms to generate)
- `app.js` (100 KB, 20ms to generate)
- `logo.png` (50 KB, 5ms to generate)
- `hero.jpg` (200 KB, 5ms to generate)

HTTP/1.1 Behavior

```
Timeline (milliseconds)
```

```
0ms         50ms        60ms        80ms        85ms        90ms
|           |           |           |           |           |
├───────────┼───────────┼───────────┼───────────┼───────────┤

Request 1: index.html
|──────────────────────────|
    50ms (waiting)      ↓ Response 1

                        Request 2: styles.css
                        |────────|
                          10ms   ↓ Response 2

                                  Request 3: app.js
                                  |────────────────────|
                                        20ms         ↓ Response 3

                                                     Request 4: logo.png
                                                     |───|
                                                      5ms ↓ Response 4

                                                         Request 5: hero.jpg
                                                         |───|
                                                          5ms ↓ Response 5

TOTAL TIME: 50 + 10 + 20 + 5 + 5 = 90ms
```

Problem: Head-of-Line Blocking

Even if `styles.css` is ready in 10ms, it can't be sent until `index.html` response is complete. Each request blocks the next one.

HTTP/2 Behavior

```
Timeline (milliseconds)
0ms                                            50ms
|                                              |
├──────────────────────────────────────────────┤

All requests sent at once (0ms):
→ Request 1: index.html
→ Request 2: styles.css
→ Request 3: app.js
→ Request 4: logo.png
→ Request 5: hero.jpg

Responses arrive as ready (interleaved on SAME connection):
```

```
0ms────5ms────10ms────20ms──────────────────50ms
       │      │       │                     │
       │      │       │                     └─ index.html arrives (slowest)
       │      │       └─ app.js arrives
       │      └─ styles.css arrives
       └─ logo.png & hero.jpg arrive

TOTAL TIME: 50ms (time of slowest resource only!)
```

What "Responses Arrive Together" Actually Means

It doesn't mean all responses arrive at the exact same millisecond. It means:

1. Single Connection, Multiple Streams

HTTP/2 creates streams within one TCP connection. Each request/response pair is a stream.

```
┌──────────────────────────────────────────────────────────┐
│              ONE TCP CONNECTION                          │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  │
│  │ Stream 1│  │ Stream 2│  │ Stream 3│  │ Stream 4│  │
│  │ HTML    │  │ CSS     │  │ JS      │  │ Image   │  │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘  │
└──────────────────────────────────────────────────────────┘
```

2. Interleaved Frames

Responses are broken into small frames. Frames from different streams are interleaved:

```
Wire (what actually travels through the network):

[HTML frame 1][CSS frame 1][JS frame 1][HTML frame 2][Image frame 1]
[CSS frame 2][HTML frame 3][JS frame 2][Image frame 2][CSS frame 3]...

                    All mixed together!
```

The browser reassembles frames into complete responses.

3. No Blocking

If `index.html` takes 50ms to generate, the server can still send `styles.css` (ready in 10ms) immediately without waiting for HTML.

```
HTTP/1.1:
Server: "CSS is ready... but I can't send it. Still generating HTML... "

HTTP/2:
Server: "CSS is ready! Sending it now. HTML still cooking, will send when done.
"
```

Real-World Example: Loading Twitter/X Homepage

Resources to Load:

Resource	Size	Server Time
main.html	50 KB	100ms
bundle.js	500 KB	20ms
styles.css	100 KB	10ms
twitter-logo.svg	5 KB	5ms
profile-pic.jpg	30 KB	50ms (from CDN)

HTTP/1.1 (with 6 parallel connections - browser limit):

Browsers work around HTTP/1.1 by opening multiple connections (typically 6 per domain):

```
Connection 1: main.html ──────────────────────────────> (100ms)
Connection 2: bundle.js ──────────> (20ms) [IDLE........]
Connection 3: styles.css ────> (10ms) [IDLE............]
Connection 4: logo.svg ──> (5ms) [IDLE................]
Connection 5: profile.jpg ────────────────> (50ms) [IDLE]
Connection 6: [IDLE...............................]

Total: 100ms (limited by slowest + connection overhead)
BUT: 6 TCP connections = 6x handshake overhead!
```

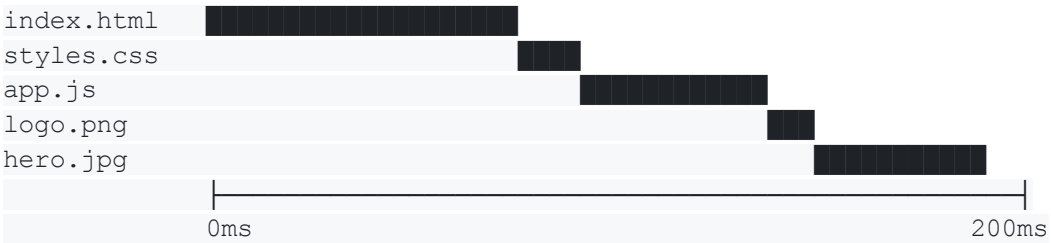
HTTP/2 (single connection):

```
Single Connection with Multiplexing:

                      0ms     10ms    20ms    50ms    100ms
styles.css            |───────|
logo.svg              |─────|
bundle.js             |──────────────|
profile.jpg           |──────────────────────────|
main.html             |──────────────────────────────────────────────|

All streams on ONE connection!
Total: 100ms (just the slowest resource)
Overhead: 1 TCP handshake instead of 6!
```

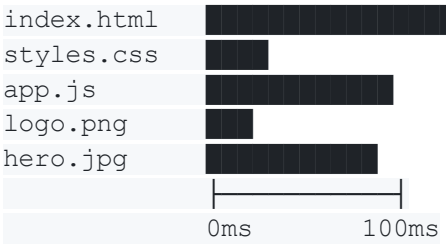
Visual Comparison: Network Waterfall

This is what you'd see in Chrome DevTools:

HTTP/1.1 Waterfall (Staircase Pattern):



HTTP/2 Waterfall (Parallel Pattern):



Why This Matters in System Design

1. Fewer Connections = Better Performance

Protocol	Connections Needed	TCP Handshakes
HTTP/1.1	6+ per domain	6+ (expensive!)
HTTP/2	1 per domain	1

2. Server Resource Usage

```
HTTP/1.1 with 1000 users, 6 connections each:
= 6,000 TCP connections to manage

HTTP/2 with 1000 users, 1 connection each:
= 1,000 TCP connections to manage
```

3. No More Domain Sharding Hacks

In HTTP/1.1 era, developers used tricks like:

```
images1.example.com  (6 connections)
images2.example.com  (6 connections)
images3.example.com  (6 connections)
= 18 parallel connections!
```

With HTTP/2, this is counterproductive because each domain requires a new TCP connection. Just use one domain.

Summary

Aspect	HTTP/1.1	HTTP/2
Request handling	One at a time per connection	Multiple simultaneously
Connection usage	Multiple connections needed	Single connection sufficient
Response delivery	Must wait for previous	Sent immediately when ready

Blocking	Head-of-line blocking	No blocking (streams independent)
Real-world analogy	Single checkout lane	Self-checkout with many stations

"All responses arrive together" means they travel on the same connection simultaneously, arriving as soon as each is ready - not waiting in line behind slower resources.

---

4. DNS - Domain Name System

What is DNS?

DNS is like the phonebook of the internet. It translates human-readable domain names to IP addresses.

```
www.google.com  →  DNS  →  142.250.190.68
(Domain Name)            (IP Address)
```

Why? Because humans remember names, computers need numbers.

DNS Hierarchy

DNS is a hierarchical, distributed system:

```
                    ┌─────────────────┐
                    │   Root DNS (.)  │  ← 13 root servers worldwide
                    └─────────────────┘
                            │
        ┌───────────────────┼───────────────────┐
        ▼                   ▼                   ▼
   ┌─────────┐         ┌─────────┐         ┌─────────┐
   │  .com   │         │  .org   │         │  .io    │  ← TLD Servers
   └─────────┘         └─────────┘         └─────────┘
        │
        ▼
   ┌──────────────┐
   │ google.com   │  ← Authoritative Nameserver
   │ (NS records) │
```

```
    └──────────┬──────────┘
               │
               ▼
    ┌──────────────────┐
    │ www.google.com│  ← A Record (IP address)
    │ 142.250.190.68│
    └──────────────────┘
```

DNS Query Process (Step by Step)

When you type `www.example.com`:

```
┌─────────────────────────────────────────────────────────────────┐
│ 1. Browser Cache                                                │
│    Check: Does browser already know the IP?                     │
│    If yes → Use cached IP                                       │
│    If no → Continue                                             │
├─────────────────────────────────────────────────────────────────┤
│ 2. OS Cache                                                     │
│    Check: Does the operating system have it cached?             │
│    If yes → Use cached IP                                       │
│    If no → Continue                                             │
├─────────────────────────────────────────────────────────────────┤
│ 3. Router Cache                                                 │
│    Your home router might have it cached                        │
├─────────────────────────────────────────────────────────────────┤
│ 4. ISP's DNS Resolver (Recursive Resolver)                      │
│    Your Internet Service Provider's DNS server                  │
│    If cached → Return IP                                        │
│    If not → Query authoritative servers                         │
├─────────────────────────────────────────────────────────────────┤
│ 5. Root DNS Server                                              │
│    "I don't know example.com, but .com TLD is at 192.55.83.30"  │
├─────────────────────────────────────────────────────────────────┤
│ 6. TLD DNS Server (.com)                                        │
│    "I don't know www.example.com, but example.com's             │
│     nameserver is at ns1.example.com (93.184.216.34)"           │
├─────────────────────────────────────────────────────────────────┤
│ 7. Authoritative Nameserver (example.com)                       │
│    "www.example.com is at 93.184.216.34"                        │
├─────────────────────────────────────────────────────────────────┤
│ 8. Response Cached                                              │
│    ISP resolver caches it, returns to your computer             │
│    Browser caches it for future requests                        │
└─────────────────────────────────────────────────────────────────┘
```

DNS Record Types

Record	Purpose	Example
A	Maps domain to IPv4	`example.com → 93.184.216.34`
AAAA	Maps domain to IPv6	`example.com → 2606:2800:220:1:...`
CNAME	Alias to another domain	`www.example.com → example.com`
MX	Mail server	`example.com → mail.example.com`
TXT	Text data (verification, SPF)	Domain verification
NS	Nameserver for domain	`example.com → ns1.example.com`
SOA	Start of Authority (primary info)	Zone administration info

DNS Caching and TTL

TTL (Time To Live): How long to cache a DNS record.

```
example.com    A    93.184.216.34    TTL: 3600 (1 hour)
```

- After 3600 seconds, the cached record expires
- Next request will query DNS servers again
- Trade-off:
  - Long TTL = Less DNS traffic, slower updates
  - Short TTL = More DNS traffic, faster updates

DNS in System Design

Why DNS matters for system design:

1. Load Balancing via DNS

Multiple A records can point to different servers:

```
api.example.com → 192.168.1.1
api.example.com → 192.168.1.2
api.example.com → 192.168.1.3
```

DNS rotates through them (Round Robin).

2. Geographic Load Balancing

Return different IPs based on user location:

```
User in USA     → api.example.com → 192.168.1.1 (US server)
User in Europe  → api.example.com → 10.0.0.1 (EU server)
```

3. Failover

If a server dies, update DNS to point elsewhere. (But beware of TTL - clients may cache old IP!)

---

5. TCP vs UDP

What are TCP and UDP?

Both are transport layer protocols that define how data is sent over the internet.

TCP (Transmission Control Protocol)

TCP is like a phone call - reliable, ordered, and connection-oriented.

Key Characteristics:

- Connection-oriented: Must establish connection first
- Reliable: Guarantees delivery
- Ordered: Data arrives in correct order
- Error-checked: Detects and retransmits lost packets
- Flow control: Adjusts speed to prevent overwhelming receiver

TCP 3-Way Handshake (Connection Setup)

```
Client                                        Server
   |                                            |
   |──────── SYN (seq=100) ────────────────────>|  "I want to connect"
   |                                            |
   |<──────── SYN-ACK (seq=300, ack=101) ───────|  "OK, I acknowledge"
   |                                            |
   |──────── ACK (ack=301) ────────────────────>|  "Great, let's talk"
   |                                            |
   |           CONNECTION ESTABLISHED           |
```

TCP Data Transfer

```
Client                                      Server
   |                                           |
   |──────── Data Packet 1 ──────────────────>|
   |<─────── ACK ────────────────────────────|  "Got it"
   |                                           |
   |──────── Data Packet 2 ──────────────────>|
   |         (lost in transit)                 |
   |         ... timeout ...                   |
   |──────── Data Packet 2 (retransmit) ──────>|  Sender retries
   |<─────── ACK ────────────────────────────|  "Got it"
```

TCP 4-Way Handshake (Connection Teardown)

```
Client                                      Server
   |                                           |
   |──────── FIN ────────────────────────────>|  "I'm done sending"
   |<─────── ACK ────────────────────────────|  "OK"
   |<─────── FIN ────────────────────────────|  "I'm done too"
   |──────── ACK ────────────────────────────>|  "OK, goodbye"
   |                                           |
   |           CONNECTION CLOSED               |
```

UDP (User Datagram Protocol)

UDP is like sending a postcard - fast, simple, but no guarantees.

Key Characteristics:

- Connectionless: No handshake needed
- Unreliable: No delivery guarantee
- Unordered: Packets may arrive out of order
- No congestion control: Sender blasts data
- Lightweight: Less overhead, faster

UDP Data Transfer

```
Client                                      Server
   |                                           |
   |──────── Data Packet 1 ──────────────────>|  (hope it arrives)
   |──────── Data Packet 2 ──────────────────>|  (no confirmation)
   |──────── Data Packet 3 ──────────────────>|  (maybe lost, oh well)
   |                                           |
   |         NO ACKNOWLEDGMENTS                |
```

TCP vs UDP Comparison

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Guaranteed delivery	Best-effort
Ordering	Ordered	No ordering
Speed	Slower (overhead)	Faster
Header Size	20-60 bytes	8 bytes
Use Case	Accuracy critical	Speed critical

When to Use TCP vs UDP

Use TCP When:

Application	Why TCP
Web browsing (HTTP/HTTPS)	Must receive complete HTML
Email (SMTP, IMAP)	Can't lose emails
File transfer (FTP)	Files must be complete
SSH, Remote access	Commands must arrive
Database connections	Data integrity critical

Use UDP When:

Application	Why UDP
Video streaming	A few lost frames are OK
Voice/Video calls (VoIP)	Real-time matters more than perfection
Online gaming	Low latency critical
DNS queries	Simple request/response
Live broadcasts	Can't wait for retransmissions

Real-World Examples

Netflix Video Streaming

- Uses TCP for the main video stream
- Why? They buffer ahead, so reliability > speed
- Lost packets would cause playback issues

Zoom Video Call

- Uses UDP for real-time video/audio
- Why? Better to skip a frame than wait for retransmission
- Lost packet = tiny glitch, retransmit = noticeable delay

Online Gaming (Fortnite, CS:GO)

- Uses UDP for game state
- Position updates sent 60+ times/second
- Old position data is useless - just send the new one

DNS

- Uses UDP (port 53) for queries
- Simple: one question, one answer
- Falls back to TCP for large responses (zone transfers)

Ports

Both TCP and UDP use ports to identify applications:

Port	Protocol	Service
20, 21	TCP	FTP
22	TCP	SSH
23	TCP	Telnet
25	TCP	SMTP
53	UDP/TCP	DNS
80	TCP	HTTP
443	TCP	HTTPS

3306	TCP	MySQL
5432	TCP	PostgreSQL
6379	TCP	Redis
27017	TCP	MongoDB

Practice Questions

Test your understanding with these questions:

Conceptual Questions:

1. Why do we need system design? What problems does it solve?
2. Explain the difference between stateful and stateless servers.
3. Why is HTTPS important? What happens in a TLS handshake?
4. Trace the complete journey of an HTTP request from browser to server.
5. What's the difference between 401 and 403 status codes?
6. Explain DNS resolution step by step.
7. When would you choose UDP over TCP?

Scenario Questions:

1. You're building a live sports score app. Would you use TCP or UDP? Why?
2. A user reports they can access `old-url.com` but get "Not Found" on `new-url.com`. The DNS was updated 2 hours ago. What's the likely issue?
3. Your API returns 200 OK but the response body is empty. What's wrong?
4. Design a simple URL shortener - what components would you need?

Key Takeaways

1. System Design is about making trade-offs to build scalable, reliable systems
2. Client-Server is the foundation - understand request/response flow
3. HTTP is stateless - each request is independent
4. HTTPS encrypts data using TLS - always use in production
5. DNS translates domains to IPs through a hierarchical system
6. TCP = reliable but slower; UDP = fast but unreliable

7. Choose the right protocol based on your requirements